

```

// Receive the MathResult object
MathResult result = did(x, y);

// Access results by their field names
System.out.println("Sum: " + result.sum());
System.out.println("Product: " + result.product());

sc.close();
}

// Return type is 'MathResult'
public static MathResult did(float a, float b) {
    return new MathResult((a + b), (a * b));
}
}

```

User prompt: can you create a guide which is explaining it properly i will make its pdf and save it for future , give proper guide including all the ways , static and not static method and everything

Response: I will now generate a comprehensive, print-ready reference guide covering all methods for returning multiple values in Java, including static versus instance (non-static) approaches, memory execution models, and modern best practices.

What is covered in this guide:

- 1. Root-Cause Analysis:** Explains why `return new ((a+b),(a*b));` fails to compile and breaks down the JVM's single-value return contract.
 - 2. Memory Model & Inline Allocation:** Illustrates how the JVM stack and heap interact when returning composite references, clarifying why `return new ...` works without extra temporary variables.
 - 3. Core Solutions:**
 - **Primitive Arrays:** Direct, index-based returns for same-type values.
 - **Java 16+ Records:** Modern, immutable data carriers with named field accessors.
 - **POJO / Custom Classes:** Backward-compatible class design for older Java versions.
 - **Collections & Lists:** Solutions for variable-length outputs.
 - 4. Static vs. Non-Static (Instance) Methods:** Clear differentiation between class-level pure methods and object-level stateful methods.
 - 5. Selection Matrix:** A comparison table evaluating type flexibility, performance, and idiomatic use cases.
-

User prompt: nope i donnt mean this i mean , how to return multiple value in return state , different ways to do it and what will be different in this if method is static and non static ,

Response: Comprehensive Guide: Returning Multiple Values in Java

1. The Core Problem & What Java Actually Does

In languages like Python or Go, you can return multiple values natively using syntax like:

```
# Python tuple return
return a + b, a * b
```

In Java, this **does not exist**. When you wrote:

```
// ❌ COMPILER ERROR
return new ((a + b), (a * b));
```

the compiler rejected it because:

- 1. The JVM Return Contract:** In bytecode, every method return instruction (`ireturn`, `freturn`, `areturn`, etc.) can place exactly **one** item onto the execution stack.

2. **new requires a concrete type:** new tells the JVM: *"Allocate memory on the heap for an instance of class X or an array of type Y[]."* You cannot place raw parentheses (...) after new because Java has no built-in Tuple class or parenthetical syntax for tuples.

The Solution: Bundling

To return multiple values, you must pack them into a **single container** (an array, a record, or an object) on the heap, and return the **single reference** pointing to that container.

2. All Practical Ways to Return Multiple Values

Way 1: Returning an Array (Best for Identical Primitive Types)

Use this when all returned values share the **same data type** (e.g., all float, all int) and their order is obvious.

Method Definition:

```
public static float[] calculate(float a, float b) {
    // Allocates a 2-element array and returns its reference in one step
    return new float[] { (a + b), (a * b) };
}
```

How the Caller Uses It:

```
float[] result = calculate(10.0f, 5.0f);
```

```
float sum = result[0];    // 15.0
float product = result[1]; // 50.0
```

- **Pros:** Extremely fast; no need to define separate classes or records; zero boilerplate.
- **Cons:** Fragile. You have to remember whether index [0] was the sum or the product. Can only hold one data type (e.g., all float).

Way 2: Using a record (Modern Java 16+ Standard)

A record is a special kind of class created specifically to act as an immutable data container. It automatically provides constructors, field getters, toString(), equals(), and hashCode() with zero boilerplate.

Defining the Record & Method:

```
// Define the container (can be inside or outside your class)
public record CalculationResult(float sum, float product) {}

public static CalculationResult calculate(float a, float b) {
    // Instantiates the record directly in the return line
    return new CalculationResult(a + b, a * b);
}
```

How the Caller Uses It:

```
CalculationResult result = calculate(10.0f, 5.0f);
```

```
// Access values via named methods (same as field names):
float sum = result.sum();
float product = result.product();
```

- **Pros:** Type-safe, holds mixed data types (e.g., float, String, boolean), self-documenting names (result.sum() vs result[0]).
- **Cons:** Requires Java 16 or newer.

Way 3: Using a Traditional Class / POJO (For Java 8 and Older)

If you are working on an older code base without record support, create a small helper class.

Defining the Class & Method:

```
class MathResult {
    public final float sum;
    public final float product;
}
```

```
    public MathResult(float sum, float product) {
        this.sum = sum;
        this.product = product;
    }
}

public static MathResult calculate(float a, float b) {
    return new MathResult(a + b, a * b);
}
```

How the Caller Uses It:

```
MathResult result = calculate(10.0f, 5.0f);
System.out.println(result.sum);
System.out.println(result.product);
```

Way 4: Using Collections (List or Map)

A. List (For dynamic lengths of the same type)

```
import java.util.List;

public static List<Float> calculate(float a, float b) {
    return List.of(a + b, a * b);
}
```

- **Caller:** `List<Float> res = calculate(x, y); float sum = res.get(0);`
- **Note:** Forces primitive float to auto-box into Float objects, which consumes slightly more memory.

B. Map (For ad-hoc key-value pairs)

```
import java.util.Map;

public static Map<String, Float> calculate(float a, float b) {
    return Map.of(
        "sum", a + b,
        "product", a * b
    );
}
```

- **Caller:** `float sum = res.get("sum");`

3. What Changes If the Method is static vs. Non-static?

The short answer: **The way you return multiple values inside the method does NOT change at all.**

What changes is **how the method is called** and **what data the method has access to**.

Comparison Summary

Feature	static Method	Non-static (Instance) Method
Ownership	Belongs to the Class itself .	Belongs to a specific object instance .
How it's called	Directly via class name: <code>MyClass.did(a, b)</code>	Requires new: <code>MyClass obj = new MyClass(); obj.did(a, b)</code>
Can access instance variables (this)?	❌ No. Cannot read fields of specific instances.	✅ Yes. Can use internal object state.
Return syntax (return new ...)	Identical	Identical

Side-by-Side Code Example

```
package REPO.DSA.JAVA.Lec_12_Method;

public class Calculator {

    // An instance variable (only accessible by non-static methods)
```

```

private float offset = 1.0f;

// -----
// 1. STATIC METHOD
// -----
// Needs ALL inputs passed in as parameters.
// It does not depend on any specific object's state.
public static float[] calculateStatic(float a, float b) {
    return new float[]{ (a + b), (a * b) };
}

// -----
// 2. NON-STATIC (INSTANCE) METHOD
// -----
// Can combine passed-in parameters WITH internal instance variables.
public float[] calculateInstance(float a, float b) {
    // 'offset' is available here via the current object
    return new float[]{ (a + b + this.offset), (a * b) };
}

// -----
// MAIN EXECUTION
// -----
public static void main(String[] args) {
    float x = 4.0f;
    float y = 2.0f;

    // --- CALLING STATIC ---
    // Notice: NO object is created to call this method.
    float[] staticRes = Calculator.calculateStatic(x, y);
    System.out.println("Static Sum: " + staticRes[0]);           // 6.0
    System.out.println("Static Product: " + staticRes[1]);       // 8.0

    // --- CALLING NON-STATIC ---
    // Notice: You MUST create an instance with 'new' first.
    Calculator calc = new Calculator();
    float[] instanceRes = calc.calculateInstance(x, y);
    System.out.println("Instance Sum: " + instanceRes[0]);       // 7.0 (includes offset)
    System.out.println("Instance Product: " + instanceRes[1]);   // 8.0
}
}

```

4. Complete, Ready-to-Run Code for Your Project

Here is your exact file updated with correct types, correct inputs, clean syntax, and both options demonstrated:

```

package REPO.DSA.JAVA.Lec_12_Method;

import java.util.Scanner;

public class multipleReturnValue {

    // 1. Record definition (can hold multiple named values)
    public record OperationResult(float sum, float product) {}

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter two floats: ");
        float x = sc.nextFloat();
        float y = sc.nextFloat();

        // Calling Option A (Array return)
        float[] arrayResult = didAsArray(x, y);
        System.out.println("[Array] Sum: " + arrayResult[0] + ", Product: " + arrayResult[1]);

        // Calling Option B (Record return)
        OperationResult recordResult = didAsRecord(x, y);
        System.out.println("[Record] Sum: " + recordResult.sum() + ", Product: " + recordResult.product());

        sc.close();
    }

    // Return using an Array (same type)

```

```
public static float[] didAsArray(float a, float b) {  
    return new float[]{ (a + b), (a * b) };  
}  
  
// Return using a Record (named fields, allows mixed types)  
public static OperationResult didAsRecord(float a, float b) {  
    return new OperationResult((a + b), (a * b));  
}  
}
```

5. Quick Decision Cheat-Sheet

- Need to return **2–3 numbers of the same type** quickly in competitive programming or DSA?

👉 `return new float[]{val1, val2};`

- Need to return **multiple values with clear names or different types** (e.g., `int count`, `String status`)?

👉 `return new MyRecord(val1, val2);`

- Does the method need to use fields stored on an object?

👉 Make it **non-static** (`calc.did(...)`).

- Does the method just take inputs, compute, and return results without needing object state?

👉 Make it **static** (`Calculator.did(...)`).
